



12. Modelagem Discreta Orientada a Eventos

12.1 Introdução

Nos capítulos anteriores foram apresentados paradigmas de modelagem particularmente adequados a diferentes classes de sistemas: equações diferenciais para dinâmicas contínuas, máquinas de estados finitos para comportamento lógico discreto, modelos híbridos para sistemas ciberfísicos e autômatos celulares para sistemas espacialmente distribuídos. Entretanto, há uma classe muito ampla de sistemas em engenharia, computação, logística, serviços, saúde, manufatura e operações em geral cuja dinâmica é mais naturalmente descrita não por evolução contínua do estado nem por atualização espacial uniforme, mas por uma sequência de ocorrências discretas relevantes no tempo. Nesses sistemas, o que altera o estado do modelo não é simplesmente a passagem uniforme do tempo, mas a ocorrência de chegadas, inícios de atendimento, termos de processamento, liberações de recursos, falhas, reparos, inspeções, decisões de roteamento e outros fatos pontuais capazes de modificar a situação do sistema. Esse é o domínio natural da **Simulação Discreta Orientada a Eventos** (*Discrete-Event Simulation* – DES, ou, na terminologia adotada neste livro, *SDOE*).

A ideia central da SDOE é conceitualmente simples, mas metodologicamente muito poderosa: o estado do sistema só precisa ser atualizado quando algum **evento** relevante ocorre.

Entre dois eventos consecutivos, assume-se que nada de essencial acontece do ponto de vista do modelo, ou, mais precisamente, que nenhuma variável de estado que participe da lógica simulada sofre alteração que exija processamento explícito. Essa observação permite representar de forma eficiente sistemas com filas, competição por recursos, lógica de roteamento, reprocessamentos, inspeções, prioridades, lotes, atrasos, interrupções e uma grande variedade de comportamentos operacionais típicos de processos reais.

A SDOE constitui um dos paradigmas mais importantes da modelagem e simulação aplicada. Sua relevância decorre tanto de seu alcance prático quanto de sua clareza conceitual. Linhas de produção, oficinas, hospitais, centrais de atendimento, redes logísticas, portos, aeroportos, armazéns, datacenters, sistemas operacionais, redes de comunicação, cadeias de suprimento e processos administrativos podem frequentemente ser descritos de forma natural por fluxos de entidades que percorrem componentes do modelo, disputam recursos escassos, aguardam em filas e produzem medidas de desempenho como tempo de espera, tempo no sistema, utilização de recursos, vazão e nível de serviço.

Contudo, justamente por sua forte aplicabilidade, a SDOE é frequentemente apresentada de modo excessivamente instrumental, como se bastasse aprender a arrastar módulos em um simulador. Esse caminho é insuficiente para um livro universitário. Antes de utilizar qualquer ferramenta, é necessário compreender com precisão quais são os elementos conceituais da modelagem discreta, quais hipóteses estão embutidas em sua formulação, como o tempo simulado evolui, quais estruturas são necessárias para representar filas e recursos, como estatísticas são produzidas e por que certos sistemas são adequados a esse paradigma e outros não. Só depois disso faz sentido passar à construção prática de modelos executáveis.

Assim, este capítulo foi organizado em dois grandes eixos, fortemente conectados. No primeiro, são apresentados os princípios fundamentais da SDOE: eventos, entidades, atributos, variáveis, recursos, filas, atividades, períodos de espera, relógio da simulação, calendário de eventos, replicações, medidas de desempenho e as duas perspectivas clássicas de formulação, orientada a eventos e orientada a processos. No segundo eixo, esses conceitos são traduzidos em modelos executáveis, com ênfase principal no Arena e uso complementar e pontual do OpenModelica, mostrando como um modelo conceitual discreto é mapeado para uma representação computacional concreta.

É importante delimitar o escopo deste capítulo. O foco aqui é a **modelagem discreta básica e parte da modelagem intermediária**. Portanto, serão tratados os blocos conceituais essenciais da SDOE e exemplos suficientemente ricos para introduzir práticas realistas de modelagem com chegadas, atribuição de propriedades a entidades, uso de recursos, filas, decisões, conjuntos de recursos e coleta estatística. Por outro lado, estruturas mais avançadas, modelos maiores e padrões operacionais mais elaborados serão deixados para material complementar em anexo, preservando a proporcionalidade do capítulo em relação aos demais do livro.

12.2 Natureza da Simulação Discreta Orientada a Eventos

A **simulação discreta orientada a eventos** é um paradigma de modelagem em que a evolução do estado do sistema é representada por uma sequência ordenada de ocorrências discretas no tempo. Cada ocorrência, ou **evento**, pode alterar uma ou mais variáveis de estado do sistema. Em consequência, o tempo simulado não precisa avançar continuamente nem ser percorrido em pequenos incrementos uniformes; ele pode ser atualizado diretamente para o instante do próximo evento relevante.

Essa característica distingue a SDOE de modelos contínuos, em que o estado evolui segundo equações diferenciais, e também de modelos discretos síncronos, como certos autómatos celulares, nos quais toda a estrutura do modelo é atualizada simultaneamente a cada passo. Na SDOE, o avanço temporal é **assíncrono e dirigido pela agenda dos eventos**. Se não ocorre nenhum evento entre os instantes t_i e t_{i+1} , então, do ponto de vista do modelo, não há motivo para realizar processamento intermediário. Essa economia semântica é uma das principais razões pelas quais a SDOE é tão eficiente e tão amplamente utilizada.

Há duas formas complementares de compreender esse paradigma. A primeira é a visão **orientada a eventos**, em que se pensa explicitamente no conjunto de eventos possíveis e nas ações que cada um deles dispara ao ocorrer. A segunda é a visão **orientada a processos**, em que se pensa no fluxo de entidades através do sistema, passando por componentes ou etapas que representam comportamentos do processo real. Embora essas duas visões pareçam diferentes, ambas descrevem o mesmo sistema sob perspectivas distintas. Simuladores como o Arena privilegiam fortemente a construção orientada a processos, mas sua execução interna continua sendo, em essência, orientada ao processamento ordenado de eventos.

Em termos formais, pode-se dizer que um modelo discreto orientado a eventos é composto por:

- um conjunto de **variáveis de estado**, que descrevem a situação corrente do sistema;
- um conjunto de **tipos de evento**, cada qual associado a regras de transformação do estado;
- uma estrutura de dados que represente os **eventos futuros** agendados;
- um **relógio de simulação**, que indica o instante corrente no tempo simulado;
- mecanismos de coleta e atualização de **estatísticas**, para produzir medidas de desempenho.

Em muitos sistemas reais, essa formulação é mais natural do que alternativas contínuas ou excessivamente agregadas. Um hospital, por exemplo, não muda seu estado de forma contínua em toda parte; seu comportamento relevante muda quando um paciente chega, quando inicia triagem, quando entra em consulta, quando um leito é liberado ou quando um exame fica pronto. Da mesma forma, uma linha de produção muda de estado quando um item é criado, entra numa máquina, termina processamento, falha em inspeção, segue para retrabalho ou deixa o sistema. Nesses contextos, o paradigma da SDOE não é apenas uma escolha computacional conveniente; ele corresponde à própria semântica operacional do sistema.

12.3 Conceitos Fundamentais da SDOE

A solidez de um modelo discreto orientado a eventos depende da correta compreensão de seus elementos básicos. Esses elementos não são meras convenções terminológicas de um software específico; eles constituem a estrutura conceitual do paradigma. Sem essa base, o modelador corre o risco de construir fluxogramas executáveis, mas semanticamente frágeis, inconsistentes ou incapazes de responder à pergunta de interesse do estudo.

12.3.1 Estado do sistema

O **estado do sistema** é o conjunto mínimo de informações necessárias para descrever, em um dado instante do tempo simulado, a situação relevante do modelo. Isso pode incluir, por exemplo, o número de entidades em fila, a ocupação de recursos, o conteúdo de buffers, a posição ou tipo de uma entidade, o valor de variáveis globais, o estágio operacional de um processo e outros elementos cuja alteração afete a dinâmica futura do sistema.

A definição do estado é uma decisão de modelagem. Um estado excessivamente pobre pode omitir mecanismos importantes; um estado excessivamente rico pode tornar o modelo desnecessariamente complexo. Em simulação, o estado não precisa ser apenas matematicamente suficiente; ele deve ser operacionalmente útil, isto é, suficiente para que o modelo decida corretamente o que fazer quando um evento ocorre.

12.3.2 Eventos

Um **evento** é uma ocorrência instantânea que pode alterar o estado do sistema. O termo “instantânea” não significa que o fenômeno físico real não possua duração, mas sim que, do ponto de vista da agenda da simulação, o evento é tratado como um marco temporal pontual no qual certas ações são executadas.

Exemplos típicos de eventos incluem chegada de uma entidade, início de processamento, término de processamento, entrada ou saída de fila, liberação de recurso, falha de equipamento, fim de reparo, inspeção concluída e decisão de roteamento. A ocorrência de um evento pode gerar outros eventos futuros. Por exemplo, quando uma entidade entra em um posto de atendimento e ocupa um recurso, o modelo normalmente agenda o término desse atendimento para algum instante futuro.

É essencial distinguir **evento** de **atividade**. Uma atividade possui duração; um evento ocorre em um instante. O término de uma atividade é frequentemente modelado como um evento. Em um atendimento de duração 5 minutos, por exemplo, a atividade é o próprio atendimento; o evento relevante para a agenda pode ser o instante em que esse atendimento termina.

12.3.3 Entidades

Uma **entidade** é um objeto conceitual que se move, espera, é processado, é transformado ou, de alguma forma, participa do fluxo dinâmico do sistema. Dependendo do problema, entidades podem representar clientes, peças, chamadas telefônicas, ordens de serviço, pacotes de dados, pacientes, documentos, contêineres, veículos ou qualquer elemento cuja trajetória no processo seja relevante.

A entidade não é apenas um marcador de presença no sistema. Em muitos modelos, ela carrega informação própria, é classificada em tipos, assume tempos de serviço específicos, segue rotas distintas e participa de decisões condicionais. Portanto, entidades são unidades dinâmicas centrais da SDOE. Um mesmo sistema pode conter várias classes de entidades, cada qual com comportamento, parâmetros ou prioridades próprios.

12.3.4 Atributos

Os **atributos** são valores associados a entidades individuais. Eles funcionam como propriedades locais das entidades e permitem que duas entidades do mesmo tipo tenham comportamentos diferentes ao longo da simulação. Um atributo pode representar, por exemplo, o tipo de peça, a hora de chegada, o tempo de serviço a ser usado adiante, a prioridade do cliente, o número de itens de um lote, o resultado de uma inspeção anterior ou um identificador de rota.

A distinção entre entidade e atributo é fundamental. A entidade é o objeto dinâmico que percorre o sistema; o atributo é uma informação armazenada nessa entidade. Em termos de modelagem, essa distinção permite descrever heterogeneidade entre entidades sem multiplicar desnecessariamente o número de fluxos ou estruturas paralelas do modelo.

12.3.5 Variáveis globais

Além dos atributos, modelos discretos frequentemente usam **variáveis globais** para representar informações pertencentes ao sistema como um todo, e não a uma entidade individual. Essas variáveis podem armazenar contadores, acumuladores, indicadores intermediários, estados agregados de controle ou parâmetros operacionais alterados dinamicamente.

Exemplos comuns incluem número total de entidades processadas, quantidade atual de itens em determinado estoque, contador de falhas, bandeira lógica de sistema em modo degradado, número de rejeições observadas ou soma acumulada de determinado custo. Enquanto atributos distinguem entidades entre si, variáveis globais descrevem a condição do sistema ou de subsistemas inteiros.

12.3.6 Recursos

Um **recurso** representa capacidade limitada necessária para a execução de alguma atividade. Máquinas, operadores, leitos, veículos, canais de comunicação, servidores computacionais e estações de inspeção podem ser modelados como recursos. Um recurso tem tipicamente uma

capacidade finita, e entidades precisam capturar parte dessa capacidade para serem atendidas ou processadas.

Quando um recurso não está disponível, a entidade não pode prosseguir imediatamente e normalmente entra em espera, muitas vezes em uma fila. Por isso, recursos são elementos centrais na geração de congestionamento, tempos de espera e utilização do sistema. A correta modelagem de recursos é decisiva porque grande parte das perguntas feitas em estudos de simulação gira precisamente em torno de sua capacidade, utilização, gargalos, balanceamento e disponibilidade.

12.3.7 Filas

Uma **fila** é a estrutura conceitual em que entidades aguardam até que alguma condição de continuidade seja satisfeita, geralmente a disponibilidade de um recurso, a chegada de um lote complementar, a liberação de capacidade adiante ou a ocorrência de uma autorização lógica. A fila não deve ser entendida apenas como uma linha física de espera; ela representa qualquer acúmulo ordenado de entidades em estado de espera.

Filas podem seguir diferentes disciplinas de atendimento, como FIFO (*first in, first out*), LIFO, prioridade, menor tempo primeiro ou outras regras. A disciplina adotada altera não apenas o comportamento operacional do sistema, mas também suas medidas de desempenho e sua interpretação gerencial. Em muitos modelos básicos, assume-se FIFO, mas essa escolha nunca deve ser tratada como neutra ou automática.

12.3.8 Atividades e períodos de espera

Na literatura clássica de simulação discreta, é usual distinguir **atividade** de **período de espera**. Uma atividade é uma etapa cuja duração é conhecida ou sorteada quando ela se inicia. Um período de espera, por sua vez, é uma permanência em espera cuja duração não pode ser determinada antecipadamente, porque depende da ocorrência futura de algum evento externo ao objeto que espera.

Quando uma peça entra em processamento com tempo de usinagem já sorteado, isso corresponde a uma atividade. Quando essa mesma peça aguarda até que uma máquina fique livre, isso corresponde a um período de espera. Essa distinção é útil porque revela duas fontes distintas de permanência no sistema: processamento e congestionamento.

12.3.9 Relógio da simulação

O **relógio da simulação** é a variável que armazena o instante corrente do tempo simulado. Ele não mede tempo de execução computacional do programa, mas sim o tempo interno do modelo. Seu valor é atualizado à medida que a simulação avança para o próximo evento relevante.

Uma das diferenças mais importantes entre SDOE e outras formas de simulação é que o relógio não precisa aumentar uniformemente de pequeno em pequeno passo. Em vez disso,

ele é normalmente ajustado diretamente para o instante do próximo evento agendado. Essa política de avanço torna a simulação eficiente e coerente com a semântica do paradigma.

12.3.10 Calendário de eventos

O **calendário de eventos** é a estrutura que armazena os eventos futuros já agendados, ordenados por instante de ocorrência. Sempre que uma atividade se inicia, o modelo frequentemente agenda seu término no calendário. Sempre que uma chegada é gerada, outra chegada futura pode ser programada. Assim, o calendário constitui a agenda dinâmica da simulação.

Embora o funcionamento interno dessa estrutura seja tópico mais próprio de implementação de simuladores, é conceitualmente importante entender seu papel: ele é o mecanismo que permite ao modelo “saber” qual será o próximo fato relevante no tempo simulado.

12.3.11 Replicações e estatísticas

Como muitos modelos discretos possuem aleatoriedade nas entradas ou na lógica, uma única execução do modelo raramente basta para caracterizar seu comportamento. Por isso, é comum realizar múltiplas **replicações**, isto é, múltiplas execuções do modelo sob as mesmas condições estruturais, mas com diferentes amostras aleatórias, para estimar medidas de desempenho como médias, variâncias, intervalos de confiança e comparações entre alternativas.

Durante cada replicação, o modelo coleta **estatísticas** de interesse. Entre as medidas clássicas estão tempo médio em fila, tempo médio no sistema, tamanho médio de fila, utilização de recursos, número de entidades completadas, fração de perdas, taxa de retrabalho e outras grandezas compatíveis com a pergunta do estudo. A escolha das estatísticas nunca é neutra: ela precisa derivar dos objetivos do projeto de simulação.

12.4 Mecânica Conceitual da Execução de um Modelo Discreto

Mesmo quando o modelador trabalha em um ambiente gráfico como o Arena, a lógica conceitual de execução de uma simulação discreta permanece a mesma. Compreendê-la é essencial para evitar uso meramente operacional do software e para dar significado às estruturas construídas.

Em termos conceituais, uma simulação discreta típica segue o seguinte ciclo geral. Primeiro, o modelo é inicializado: relógio ajustado para o instante inicial, variáveis de estado definidas, recursos colocados em sua condição inicial, filas vazias, estatísticas zeradas e eventos iniciais inseridos no calendário. Depois, identifica-se o evento futuro mais próximo. O relógio é então avançado diretamente para o instante desse evento. Em seguida, esse evento é processado, o que pode alterar o estado do sistema, produzir estatísticas e agendar novos eventos futuros. O ciclo se repete até que uma condição de término seja atingida.

Essa descrição, embora sintética, revela uma característica central da SDOE: o modelo não “anda no tempo” por mero decurso do relógio; ele anda de evento em evento. Se entre dois eventos não há alteração relevante de estado, nada precisa ser feito. Isso torna o paradigma particularmente adequado a sistemas em que as mudanças realmente importantes são discretas e identificáveis.

12.4.1 Exemplo conceitual: fila de atendimento com um servidor

Considere um sistema simples de atendimento com chegadas aleatórias de clientes, um único atendente e disciplina FIFO. Quando um cliente chega, podem ocorrer dois casos. Se o atendente estiver livre, o cliente inicia imediatamente o atendimento, ocupando o recurso servidor e gerando um evento futuro de término de atendimento. Se o atendente estiver ocupado, o cliente entra na fila e permanece em espera até que o recurso seja liberado.

Quando ocorre o término de atendimento, o servidor é liberado. Se houver clientes aguardando na fila, o próximo cliente é retirado da fila, inicia atendimento e outro término futuro é agendado. Se a fila estiver vazia, o recurso permanece ocioso até a próxima chegada. Toda a dinâmica do sistema pode ser descrita apenas com dois tipos principais de evento: *chegada* e *término de atendimento*. Ainda assim, o modelo produz medidas ricas: tempos de espera, tamanho da fila, utilização do servidor e tempo no sistema.

Esse exemplo é elementar, mas nele já estão presentes praticamente todos os elementos fundamentais da SDOE: entidades, recurso, fila, eventos, calendário, relógio e estatísticas.

12.4.2 Orientação a eventos e orientação a processos

Na **orientação a eventos**, o modelador explicita cada tipo de evento e escreve as ações correspondentes a seu processamento. No exemplo da fila simples, haveria explicitamente o algoritmo de chegada e o algoritmo de término de atendimento. Essa abordagem é conceitualmente precisa e é muito útil para compreender a mecânica do simulador.

Na **orientação a processos**, por outro lado, o modelador pensa o sistema como um fluxo de entidades passando por etapas ou componentes. Em vez de programar diretamente “se chegou então faça isso”, ele constrói um fluxo como: criar entidade, atribuir propriedades, capturar recurso, sofrer atraso, liberar recurso, sair do sistema. Essa é a forma típica como ferramentas de alto nível como Arena organizam a modelagem.

Essas duas perspectivas não se excluem. A orientação a processos facilita a construção do modelo a partir da visão operacional do sistema. A orientação a eventos ajuda a compreender a semântica interna do que o simulador executa. Um bom modelador deve dominar ambas, ainda que utilize no dia a dia uma ferramenta mais fortemente orientada a processos.

12.5 Do Modelo Conceitual ao Modelo Executável

Nos capítulos anteriores, foi enfatizado que a construção do modelo conceitual é etapa central do projeto de simulação. No caso da SDOE, isso continua sendo plenamente verdadeiro. Um simulador não substitui o raciocínio conceitual do modelador; ele apenas oferece uma infraestrutura para executar uma representação formalizada do sistema.

Ao passar do modelo conceitual ao modelo executável, o modelador precisa decidir, entre outros aspectos:

- quais entidades existirão e quais tipos diferentes serão necessários;
- quais atributos ou variáveis devem ser armazenados;
- quais recursos serão explicitamente modelados;
- quais filas existirão e quais disciplinas adotarão;
- quais eventos ou etapas de processo precisam ser representados;
- quais saídas de interesse devem ser coletadas;
- qual nível de detalhe é suficiente para responder à pergunta do estudo.

Em Arena, essas decisões são traduzidas principalmente por módulos de fluxo e por estruturas de dados do modelo. A prática recomendada é nunca começar diretamente pela ferramenta sem antes listar o fluxo das entidades, os recursos envolvidos, os pontos de decisão, os pontos de espera e as estatísticas desejadas. A literatura da ferramenta, inclusive em exemplos um pouco mais ricos do que os introdutórios mínimos, mostra esse caminho: primeiro, segmenta-se o sistema em componentes lógicos do processo; depois, decide-se quais módulos capturam cada parte dessa lógica; e só então se implementa o modelo executável. Isso aparece claramente em modelos introdutórios em que duas chegadas distintas requerem módulos **Create** diferentes, atributos são definidos via **Assign**, tempos específicos são guardados para uso posterior, etapas são modeladas por **Process** ou por sequências **Seize-Delay-Release**, e pontos de inspeção ou desvio são implementados por **Decide**.

12.6 Introdução Prática ao Arena

O Arena é um ambiente de simulação discreta orientado a processos, amplamente utilizado em ensino e prática profissional. Em sua camada mais acessível, ele organiza a modelagem por módulos de alto nível, especialmente os do painel **Basic Process**, que capturam operações recorrentes de modelos discretos: geração de entidades, atribuição de propriedades, processamento com ou sem recursos, decisões, coleta de registros e descarte de entidades. Obras introdutórias e a própria documentação do Arena mostram que a entrada didática natural na ferramenta ocorre por esses módulos, antes de qualquer migração para níveis mais detalhados de modelagem.

É importante, contudo, interpretar corretamente o papel desses módulos. Eles não são apenas ícones de interface; cada um representa uma operação semântica sobre o fluxo das entidades e sobre o estado do sistema. O módulo **Create** representa geração de entidades

segundo alguma lógica temporal. O módulo *Assign* modifica atributos, variáveis ou outras propriedades. O módulo *Process* ou a sequência *Seize-Delay-Release* modela o uso de recursos e a duração de atividades. O módulo *Decide* introduz ramificação lógica ou probabilística. O módulo *Record* coleta estatísticas específicas. O módulo *Dispose* representa a saída de entidades do sistema.

Embora o *Process* seja bastante conveniente e concentre em um único módulo a lógica *seize-delay-release*, a própria literatura de Arena ressalta que modeladores mais experientes frequentemente recorrem às estruturas mais explícitas *Seize*, *Delay* e *Release*, por fornecer maior transparência e flexibilidade. Isso será importante neste capítulo, porque os exemplos iniciais já começarão em um nível menos elementar, tornando mais visível a semântica do modelo. :contentReference[oaicite:7]index=7

12.7 Exemplo 1: Atendimento com Classificação, Atributos e Recurso Simples

O primeiro exemplo deste capítulo não será o fluxo mínimo *Create-Process-Dispose*, pois esse padrão, embora didaticamente válido, tende a ser pobre demais para introduzir os conceitos que realmente interessam ao leitor neste ponto da obra. Em vez disso, começaremos com um modelo ligeiramente mais estruturado, contendo *Create*, *Assign*, *Seize*, *Delay*, *Release* e *Dispose*. Assim, desde o início, o modelo já explicita a existência de atributos por entidade, ocupação de recurso e duração de atividade.

Considere um pequeno posto técnico que recebe ordens de serviço de dois tipos: ordens simples e ordens complexas. As ordens chegam segundo um processo de chegadas aleatórias, são classificadas ao entrar no sistema, recebem um tempo de atendimento coerente com sua classe e então competem pelo mesmo técnico. O objetivo do estudo é estimar tempo médio no sistema, tempo médio de espera e utilização do técnico, além de verificar se a diferença entre ordens simples e complexas gera congestionamento relevante.

Do ponto de vista conceitual, o modelo envolve:

- uma entidade *OrdemServico*;
- um atributo *TipoOS*, indicando se a ordem é simples ou complexa;
- um atributo *TempoAtendimento*, que armazena o valor sorteado para uso posterior;
- um atributo *InstanteChegada*, usado para cálculo de tempo no sistema;
- um recurso *Tecnico*;
- uma fila associada ao recurso.

Em Arena, a lógica do modelo pode ser descrita como:

Create → *Assign* → *Seize* → *Delay* → *Release* → *Dispose*

O módulo *Assign* é particularmente importante aqui. Em exemplos um pouco mais ricos da literatura de Arena, ele é usado exatamente para atribuir a entidades tempos de processamento específicos e registrar o tempo corrente de chegada por meio da variável global

TNOW, para posterior medição de ciclo. Essa é uma prática extremamente útil e conceitualmente elegante: sorteia-se ou define-se cedo uma característica da entidade e essa característica acompanha a entidade ao longo do fluxo, podendo ser usada adiante no modelo. :contentReference[oaicite:8]index=8

12.7.1 Formulação conceitual

Suponha que o tempo entre chegadas siga distribuição exponencial com média de 8 minutos. Ao chegar, cada ordem é classificada como simples com probabilidade 0,7 e complexa com probabilidade 0,3. O tempo de atendimento é triangular, com parâmetros (4,6,9) minutos para ordens simples e (8,12,18) minutos para ordens complexas. O recurso Técnico possui capacidade 1.

A entidade, ao ser criada, recebe em **Assign**:

- **TipoOS**;
- **TempoAtendimento**;
- **InstanteChegada** = **TNOW**.

Depois, a entidade tenta capturar uma unidade do recurso Técnico. Se o recurso estiver ocupado, entra na fila associada. Quando a captura ocorre, a entidade sofre atraso igual ao valor armazenado em **TempoAtendimento**. Em seguida, o recurso é liberado e a entidade deixa o sistema.

12.7.2 Interpretação de modelagem

Esse exemplo, embora simples, já introduz várias decisões corretas de modelagem. Primeiro, o tipo da ordem não foi representado por dois fluxos separados desde a origem; ele foi representado por um atributo. Isso é adequado quando o comportamento estrutural é o mesmo, mudando apenas parâmetros internos ou decisões posteriores. Segundo, o tempo de atendimento foi sorteado e armazenado explicitamente como atributo, e não recalculado depois, preservando a identidade estocástica daquela entidade. Terceiro, a captura do recurso foi representada explicitamente por **Seize**, o que torna a fila e a ocupação do recurso conceitualmente visíveis.

12.7.3 Possível extensão imediata

Uma extensão natural desse modelo é incluir um módulo **Record** imediatamente antes do **Dispose**, registrando o valor **TNOW** - **InstanteChegada**. Assim, além das estatísticas automáticas de fila e recurso, o modelo passa a coletar explicitamente o tempo total no sistema de cada ordem. Esse tipo de registro é especialmente importante quando o modelador deseja controlar com precisão quais medidas serão analisadas posteriormente.

12.8 Exemplo 2: Atendimento com Decisão e Reprocessamento Elementar

O segundo exemplo acrescenta uma lógica de decisão ao modelo anterior. Agora, após o atendimento técnico, parte das ordens segue para saída definitiva e parte precisa de retrabalho simples. O objetivo é introduzir o módulo *Decide* como mecanismo de ramificação probabilística ou lógica, sem ainda transformar o modelo em um sistema excessivamente grande.

Considere que, após o atendimento principal, uma ordem tem probabilidade 0,15 de exigir retrabalho administrativo rápido, com tempo uniforme entre 2 e 4 minutos, realizado por um assistente administrativo distinto do técnico. O fluxo geral passa a ser:

Create → Assign → Seize Técnico → Delay Atendimento → Release Técnico → Decide → [saída ou retrabalho]

Se a ordem seguir para retrabalho, então passa por:

Seize Assistente → Delay Retrabalho → Release Assistente → Dispose

12.8.1 Importância didática do Decide

O módulo *Decide* é fundamental na modelagem discreta porque introduz bifurcações estruturais no fluxo. Em um sistema real, quase nunca todas as entidades seguem exatamente a mesma rota. Algumas são aprovadas, outras rejeitadas; algumas seguem para inspeção, outras para expedição; algumas precisam de retrabalho, outras não. Portanto, a noção de processo como fluxo linear único é frequentemente insuficiente, e a modelagem passa a exigir pontos de decisão.

Do ponto de vista metodológico, o *Decide* não deve ser interpretado apenas como uma “moeda probabilística”. Ele representa uma hipótese sobre o sistema. Se o desvio é probabilístico, o modelador está assumindo que, naquele nível de abstração, a decisão pode ser representada por uma chance agregada. Em outros contextos, o desvio poderia depender de atributos, de contadores, de variáveis globais ou do resultado de inspeções anteriores.

12.8.2 Estatísticas relevantes

Nesse segundo exemplo, tornam-se relevantes não apenas as estatísticas do técnico principal, mas também:

- taxa de retrabalho;
- utilização do recurso Assistente;
- aumento do tempo médio no sistema devido ao retrabalho;
- eventual mudança no gaplo do sistema.

Observe-se que mesmo um acréscimo estrutural pequeno já altera significativamente o comportamento do modelo. Isso ilustra por que a SDOE é tão apropriada para estudos de me-

lhorar de processos: pequenas mudanças na lógica operacional podem ter efeitos importantes sobre filas, utilização e tempo no sistema.

12.9 Exemplo 3: Conjuntos de Recursos, Conjuntos de Filas e Roteamento

A partir deste ponto, vale introduzir um modelo um pouco mais intermediário, mas ainda coerente com o escopo do capítulo. Muitos sistemas reais não possuem apenas um recurso homogêneo isolado; eles possuem **múltiplos recursos equivalentes ou parcialmente equivalentes**. Em Arena, isso pode ser representado por **sets de recursos** e, quando pertinente, por **sets de filas**. Essa capacidade é importante porque evita duplicação excessiva de lógica e aproxima o modelo de estruturas operacionais reais, como equipes, estações paralelas ou grupos de atendimento.

Considere um centro de triagem com dois técnicos equivalentes, TécnicoA e TécnicoB, mas com velocidades ligeiramente diferentes. As ordens chegam ao sistema, recebem classificação e então devem ser alocadas ao primeiro técnico disponível, preferencialmente segundo uma lógica de menor fila ou seleção por conjunto de recursos. Depois do atendimento, parte segue para inspeção final, e outra parte sai diretamente.

12.9.1 Estrutura conceitual

Agora o modelo pode usar:

- um Resource Set contendo TécnicoA e TécnicoB;
- um Queue Set, se se quiser acompanhar filas associadas separadamente;
- atributos para tipo de ordem e tempo de chegada;
- um ponto de decisão após o atendimento;
- uma etapa final opcional de inspeção.

O uso de conjuntos de recursos é metodologicamente importante porque representa uma classe de problema muito comum: vários servidores semelhantes atendendo a mesma demanda. Em vez de duplicar fluxos inteiros com módulos quase idênticos para cada recurso, o modelador pode expressar essa simetria estrutural de forma mais compacta e mais fiel ao sistema.

12.9.2 Questões de modelagem

Este exemplo introduz questões mais interessantes do que a simples captura de um recurso individual. Por exemplo:

- a seleção do recurso dentro do conjunto é por disponibilidade imediata, por regra fixa, por fila mínima ou por outra política?
- os dois recursos realmente podem ser tratados como equivalentes ou devem possuir

distribuições de atendimento distintas?

- a fila deve ser comum ao conjunto de recursos ou separada por recurso?
- a política de despacho altera o desempenho médio do sistema?

Essas perguntas mostram que a modelagem discreta deixa rapidamente de ser um exercício de software e passa a ser um problema de formulação operacional. A estrutura computacional é apenas a materialização de uma hipótese sobre o funcionamento real do sistema.

12.9.3 Relação com a literatura de Arena

A própria literatura de Arena evolui da construção de modelos mínimos para exemplos em que múltiplos módulos são combinados para representar chegadas distintas, atribuição de atributos, tempos específicos por entidade, decisões probabilísticas e múltiplos pontos de saída, inclusive com registros separados por classe de resultado. Essa progressão confirma que a entrada didática adequada na prática de SDOE não precisa ficar presa ao modelo trivial de servidor único sem ramificações; é perfeitamente legítimo começar, desde que com cuidado pedagógico, com exemplos que já contenham classificação, atributos, decisões e estruturas paralelas moderadas. :contentReference[oaicite:9]index=9 :contentReference[oaicite:10]index=10

12.10 Coleta de Estatísticas e Interpretação dos Resultados

Um modelo discreto não é construído apenas para “rodar”; ele é construído para produzir evidência quantitativa sobre o sistema. Portanto, a coleta de estatísticas não é uma etapa periférica, mas parte constitutiva da modelagem. O erro comum de iniciantes é tratar estatísticas como algo a ser observado apenas no final, nos relatórios automáticos do software. Esse procedimento pode ser insuficiente, pois o software só reportará automaticamente o que estiver semanticamente associado a suas estruturas padrão ou o que tiver sido explicitamente registrado.

Na prática, há três grandes grupos de medidas que costumam interessar:

1. medidas sobre entidades, como tempo no sistema, tempo em fila, número de retrabalhos e tempos por classe;
2. medidas sobre recursos, como utilização, tempo ocioso, tempo bloqueado e número de capturas;
3. medidas agregadas do sistema, como vazão, perdas, percentual de ordens reprocessadas e nível de serviço.

Em Arena, parte dessas medidas é produzida automaticamente quando recursos, filas e processos são modelados de forma convencional. Contudo, medidas mais específicas frequentemente exigem módulos Record, atributos de tempo de chegada, variáveis auxiliares ou expressões construídas explicitamente.

O modelador deve decidir desde o início quais métricas respondem à pergunta do estudo.

Se o objetivo é reduzir congestionamento, então medidas de fila e espera são centrais. Se o objetivo é avaliar capacidade produtiva, vazão e utilização podem ser mais importantes. Se o objetivo é comparar duas políticas operacionais, então as estatísticas devem ser estruturadas de modo a permitir comparação justa entre alternativas sob replicações equivalentes.

12.11 Comentário Complementar sobre OpenModelica

Embora o Arena seja a ferramenta principal deste capítulo, é útil registrar que comportamento discreto orientado a eventos também pode ser representado, em certa medida, em ambientes mais gerais de modelagem, como o OpenModelica. Em Modelica, eventos discretos podem surgir de condições, mudanças lógicas, relógios, partições síncronas e mecanismos associados à linguagem e a seus ambientes de execução. A própria especificação da linguagem inclui elementos para equações discretas, eventos, sincronização, relógios e máquinas de estados, o que mostra que o tratamento de comportamento discreto não é externo ao formalismo.

Contudo, a vocação principal do OpenModelica neste livro não é a modelagem transacional típica da SDOE, mas a modelagem física contínua e híbrida. Por isso, seu papel aqui deve ser entendido como complementar. Quando um sistema exige integração entre comportamento contínuo e lógica de eventos discretos, OpenModelica é extremamente valioso. Quando o foco é um processo operacional com entidades, filas, recursos e roteamento transacional, Arena oferece uma representação mais direta, transparente e didaticamente adequada.

Essa distinção é importante porque ajuda o leitor a compreender que a escolha de ferramenta não é questão de preferência superficial, mas de alinhamento entre paradigma de modelagem, semântica do problema e forma de experimentação desejada.

12.12 Limites do Capítulo e Ponte para o Material Complementar

Este capítulo introduziu a SDOE em dois níveis complementares: o nível conceitual do paradigma e o nível prático da modelagem básica e parte da modelagem intermediária com Arena. Foram apresentados os elementos centrais da formulação discreta orientada a eventos e, em seguida, exemplos de complexidade crescente, já incluindo atributos, decisões, uso explícito de *Seize-Delay-Release* e estruturas com múltiplos recursos.

Entretanto, a modelagem discreta realista em problemas industriais e de serviços pode exigir muito mais: lotes, separação e reunião de entidades, prioridades mais elaboradas, agendas de recursos, falhas e reparos, transferências complexas, lógica de transporte, estruturas hierárquicas maiores, controle por variáveis globais, animações operacionais e padrões avançados de coleta de dados. Para que este capítulo permaneça proporcional aos demais e preserve seu foco formativo, esses tópicos serão deixados para material complementar em anexo.

12.13 Considerações Finais

A simulação discreta orientada a eventos é um dos paradigmas mais importantes da modelagem e simulação aplicada porque permite representar, com clareza conceitual e grande efetividade computacional, sistemas cujo comportamento relevante é determinado por ocorrências discretas no tempo. Ao longo deste capítulo, mostrou-se que a SDOE não deve ser reduzida a um conjunto de comandos de software. Ela possui uma semântica própria, baseada em estado, eventos, entidades, atributos, recursos, filas, relógio e calendário, e exige decisões explícitas sobre abstração, estrutura operacional e medidas de desempenho.

Também se procurou mostrar que a passagem da teoria para a prática não precisa nem deve ser abrupta. O uso de uma ferramenta como Arena faz sentido precisamente quando o leitor já compreende o papel conceitual das estruturas que está construindo. Por isso, a parte prática foi organizada de modo a começar já com exemplos não triviais, incorporando atributos, decisões, uso explícito de recursos e estruturas paralelas moderadas. Essa escolha aproxima melhor o capítulo da realidade dos sistemas que motivam a SDOE e evita que a prática se reduza a um exercício gráfico desprovido de densidade técnica.

Com isso, o capítulo encerra a Parte III do livro, consolidando um paradigma de modelagem fortemente ligado a processos operacionais, serviços, manufatura e sistemas computacionais. Nos capítulos seguintes, o foco deixará de ser a construção de novos paradigmas e passará a recair sobre a confiabilidade metodológica do modelo já construído: sua verificação, sua validação, sua credibilidade, o planejamento de experimentos e a interpretação estatística de seus resultados.

12.14 Resumo

Seção 12.1 Introdução

- A SDOE é adequada para sistemas cujo estado muda em ocorrências discretas relevantes no tempo.
- O paradigma é especialmente útil em processos operacionais com entidades, filas, recursos e lógica de roteamento.
- A compreensão conceitual da SDOE deve anteceder o uso instrumental do simulador.

Seção 12.2 Natureza da Simulação Discreta Orientada a Eventos

- O estado do sistema só precisa ser atualizado quando um evento relevante ocorre.
- O tempo simulado avança de evento em evento, e não por incrementos uniformes arbitrários.
- A SDOE pode ser compreendida por formulação orientada a eventos ou por formulação orientada a processos.

Seção 12.3 Conceitos Fundamentais da SDOE

- Estado do sistema, eventos, entidades, atributos, variáveis globais, recursos, filas, atividades e períodos de espera formam o núcleo conceitual da SDOE.

- O relógio da simulação representa o tempo interno do modelo, e o calendário de eventos representa a agenda de ocorrências futuras.
- Replicações e estatísticas são indispensáveis para transformar execução de modelo em evidência quantitativa.

Seção 12.4 Mecânica Conceitual da Execução de um Modelo Discreto

- Uma simulação discreta típica alterna inicialização, escolha do próximo evento, avanço do relógio, processamento do evento e agendamento de novos eventos.
- O exemplo da fila com servidor único já contém os elementos essenciais da SDOE.
- Orientação a eventos e orientação a processos são perspectivas complementares do mesmo paradigma.

Seção 12.5 Do Modelo Conceitual ao Modelo Executável

- A tradução para o simulador exige decidir entidades, atributos, recursos, filas, decisões, saídas e nível de detalhe.
- O software executa uma formulação; ele não substitui a construção do modelo conceitual.

Seção 12.6 Introdução Prática ao Arena

- O Arena organiza a modelagem por módulos de alto nível como Create, Assign, Process, Decide, Record e Dispose.
- A sequência explícita Seize-Delay-Release oferece maior transparência semântica e flexibilidade em muitos modelos.

Seção 12.7 Exemplo 1

- Um modelo com Create, Assign, Seize, Delay, Release e Dispose já permite introduzir atributos, tempos por entidade e ocupação de recurso.
- Atributos como tipo de ordem, tempo de atendimento e instante de chegada tornam o fluxo mais expressivo e realista.

Seção 12.8 Exemplo 2

- O módulo Decide permite modelar desvios estruturais do fluxo, como retrabalho, inspeção ou rejeição.
- Pequenas mudanças na lógica do processo podem alterar significativamente as medidas de desempenho do sistema.

Seção 12.9 Exemplo 3

- Sets de recursos e filas permitem representar servidores paralelos e políticas de alocação sem duplicação excessiva de lógica.
- A modelagem com múltiplos recursos equivalentes introduz decisões operacionais sobre despacho, seleção e balanceamento.

Seção 12.10 Coleta de Estatísticas e Interpretação

- Estatísticas de entidades, recursos e sistema devem ser escolhidas em função dos obje-

tivos do estudo.

- Relatórios automáticos do simulador são úteis, mas frequentemente precisam ser complementados por registros explícitos.

Seção 12.11 Comentário Complementar sobre OpenModelica

- Eventos discretos também podem ser representados em ambientes como OpenModelica.
- OpenModelica é especialmente valioso quando o comportamento discreto precisa coexistir com dinâmica contínua.
- Arena continua sendo a ferramenta principal deste capítulo para modelagem transacional discreta.

Seção 12.12 Limites do Capítulo e Ponte para o Material Complementar

- Este capítulo cobre fundamentos, modelagem discreta básica e parte da modelagem intermediária.
- Estruturas mais avançadas de SDOE serão tratadas em anexo.

12.15 Exercícios Propostos

Exercício 12.1 Explique por que a SDOE é mais adequada do que um modelo de tempo contínuo para representar uma central de atendimento telefônico com filas, operadores e roteamento por tipo de chamada.

Exercício 12.2 Descreva a diferença entre entidade, atributo e variável global em um modelo de simulação discreta. Dê um exemplo de uso apropriado de cada um.

Exercício 12.3 Explique a diferença entre atividade e período de espera, e mostre como essa distinção aparece em um sistema com recurso escasso.

Exercício 12.4 Discuta a diferença entre formulação orientada a eventos e formulação orientada a processos, e explique por que ambas podem representar o mesmo sistema.

Exercício 12.5 Em um sistema com uma fila FIFO e um recurso com capacidade 2, quais elementos do estado do sistema são indispensáveis para descrever corretamente a dinâmica futura do modelo?

Exercício 12.6 Projete conceitualmente um modelo Arena para um processo com:

- chegadas exponenciais;
- duas classes de entidades;
- um recurso comum;
- tempo de atendimento diferente por classe;
- retrabalho probabilístico.

Descreva os módulos principais, os atributos necessários e as estatísticas de interesse.

Exercício 12.7 Explique por que pode ser vantajoso usar **Assign** para armazenar cedo um tempo de processamento em um atributo, em vez de gerar esse tempo apenas no módulo de atraso.

Exercício 12.8 Considere um sistema com dois atendentes equivalentes. Discuta as diferen-

ças entre:

1. modelar os dois atendentes por dois fluxos paralelos separados;
2. modelá-los por meio de um conjunto de recursos.

Exercício 12.9 Proponha uma medida de desempenho principal para cada um dos sistemas abaixo:

1. pronto atendimento hospitalar;
2. oficina com retrabalho;
3. central de suporte técnico;
4. linha de inspeção com sucata.

Justifique.

Exercício 12.10 Explique em que situações o relatório automático do simulador pode ser insuficiente, exigindo uso explícito de Record, atributos de tempo e variáveis auxiliares.

Exercício 12.11 Construa, em Arena, o modelo do Exemplo 1 deste capítulo e compare os resultados sob duas alternativas:

1. um único técnico;
2. dois técnicos equivalentes.

Discuta o impacto sobre espera média e utilização.

Exercício 12.12 Amplie o modelo do Exemplo 2 incluindo prioridade mais alta para ordens complexas no retrabalho. Discuta como essa alteração pode afetar o tempo médio no sistema para cada classe.

Exercício 12.13 Implemente um pequeno exemplo em OpenModelica ou em ambiente equivalente que contenha ao menos um evento discreto de mudança de modo operacional. Explique por que esse tipo de modelagem não substitui diretamente um simulador transacional como o Arena.

Exercício 12.14 Discuta criticamente as vantagens e limitações de começar a modelagem discreta com módulos de alto nível, como Process, em comparação com o uso explícito de Seize-Delay-Release.

Exercício 12.15 Explique por que um modelo executável e visualmente plausível ainda pode ser metodologicamente inadequado se a pergunta do estudo, as estatísticas de interesse e as hipóteses de abstração não tiverem sido bem definidas.

Base bibliográfica do capítulo. Este capítulo se apoia principalmente em [29], [6], [24] e [42] para a apresentação dos fundamentos da simulação discreta orientada a eventos, incluindo eventos, entidades, atributos, recursos, filas, calendário de eventos, replicações e interpretação dos resultados. A parte prática, com ênfase na construção de modelos básicos e intermediários em Arena, apoia-se especialmente em [24] e [42], tanto na semântica dos módulos quanto na organização de exemplos progressivamente mais ricos. Já o comentário complementar sobre eventos discretos em ambientes mais gerais de modelagem dialoga com [33], [36] e [14], preservando o papel do OpenModelica como ferramenta complementar neste contexto.

RASCUNHO